ФИНАЛ ФСП · КЕЙС СТОЛОТО

SPORTIKUS

Документация платформы

Платформа спортивных активностей: продукт, сценарии, покрытие ТЗ и устройство системы. Текст совпадает с документацией на сайте.

2026

Содержание

ГЛАВНАЯ	Sportikus — документация
ПРОДУКТ	Обзор продукта
ПРОДУКТ	Карта страниц
ПРОДУКТ	Акторы и демо-аккаунты
ПРОДУКТ	Use cases
ПРОДУКТ	Use cases пользователя
ПРОДУКТ	Use cases организатора
ПРОДУКТ	Use cases администратора
ПРОДУКТ	User flows
БИЗНЕС	Покрытие ТЗ
БИЗНЕС	Нефункциональные требования
ИНЖЕНЕРИЯ	Инженерия
ИНЖЕНЕРИЯ	Архитектура
ИНЖЕНЕРИЯ	Модель данных
ИНЖЕНЕРИЯ	API
ИНЖЕНЕРИЯ	Конструктор мероприятий
ИНЖЕНЕРИЯ	Рекомендательная система
ИНЖЕНЕРИЯ	Автопроверка мероприятий
ИНЖЕНЕРИЯ	Локальный запуск
ИНЖЕНЕРИЯ	Тестирование
ИНЖЕНЕРИЯ	Ограничения и развитие
ГЛОССАРИЙ	Глоссарий

Sportikus — документация

Документация платформы **Sportikus**: от идеи потренироваться к конкретной компании за два клика — в любом городе страны. Решение подготовлено для финала Чемпионата России по продуктовому программированию (ФСП, кейс Столото).

Разделы

Раздел	Описание
Обзор продукта	Что делает платформа, для кого и какие возможности в неё входят
Карта страниц	Экраны веб-клиента, главное действие на каждом
Акторы и демо-аккаунты	Роли, права и готовые аккаунты для проверки
Use cases	Сценарии использования по ролям
User flows	Пошаговые пути с диаграммами, включая 6 сценариев ТЗ
Покрытие ТЗ	Какие требования выполнены и как это проверить
Нефункциональные требования	Масштаб, безопасность, отказоустойчивость, переносимость
Инженерия	Архитектура, модель данных, API, рекомендации, автопроверка, запуск, тесты
Глоссарий	Термины платформы

Платформа в двух словах

Пользователь находит мероприятие на карте 2GIS или в каталоге, записывается сразу или по заявке — и получает электронный QR-билет. Организатор (человек или одобренная организация) собирает мероприятие в конструкторе из готовых шаблонов, принимает заявки, отвечает на вопросы в обсуждении и отмечает участников на входе. Администратор модерирует организации, мероприятия и комментарии, а новые мероприятия ещё до этого проходит автопроверка: фильтр запрещённых тем и, при сомнении, проверка ИИ.

Рекомендации на странице «Для вас» строятся на PostgreSQL + PostGIS, каждая карточка объясняет, почему она показана.

Быстрый старт

```
docker compose up --build -d
```

Веб-приложение — `http://localhost:3000`, Swagger UI — `http://localhost:8080/swagger-ui/index.html`. Подробности: Локальный запуск.

Обзор продукта

Sportikus помогает найти и организовать спортивное (и не только) мероприятие рядом с собой. Платформа не привязана к одному городу: каталог, геопоиск и рекомендации рассчитаны на масштаб страны.

Для кого

Аудитория	Что получает
Любитель спорта	Карту и каталог мероприятий рядом, понятные условия, запись в пару кликов, персональные рекомендации с объяснением причин
Организатор-частное лицо	Конструктор мероприятия, контроль над заявками, обсуждение под событием, электронные билеты и отметку участников на входе
Организация (клуб, секция, федерация)	Публикацию без премодерации после одобрения, блок сведений об организаторе, подписчиков, профиль с логотипом
Администратор площадки	Одобрение организаций, снятие мероприятий с публикации, скрытие комментариев, разбор помеченных автопроверкой мероприятий

Возможности

Область	Что умеет платформа
Карта и каталог	Карта 2GIS с метками (время до старта, свободные места), список, поиск, фильтры по виду спорта, уровню, режиму записи, времени, расстоянию, цене. Каталог загружается для видимой области карты
Конструктор мероприятий	Готовые шаблоны с настраиваемыми полями, свой вид спорта, свои параметры, анкета участника; для организаций — блок сведений организатора
Мероприятия от пользователя и организации	Пользователь публикует от своего имени, организация — от имени организации после одобрения администратором
Участие с подтверждением	Режимы «по заявке» и «свободный вход», отмена, исключение, лимит мест под блокировкой
Электронные билеты	У подтверждённого участника — уникальный QR-билет; организатор сканирует его камерой или вводит код вручную
Обсуждение под событием	Вопросы участников и ответы организатора со значком «Организатор», автообновление, мягкое скрытие сообщений
Рекомендации	Раздел «Для вас»: подборка по интересам, месту, времени и истории с причинами и кнопкой «Не интересно»
Социальные функции	Друзья (заявки, поиск по e-mail, «Идут друзья» на карточке), подписки на организации, достижения за активность
Архив	Организатор завершает прошедшее мероприятие — оно уходит в архив организатора и участников
Автопроверка	Фильтр запрещённых тем и проверка ИИ за несколько секунд после создания; сомнительное помечается ⚠ для администратора

Область	Что умеет платформа
Модерация	Организации, мероприятия, комментарии; фильтры «Требуют внимания» и «Отклонены автопроверкой»
Вся страна	Геопоиск PostGIS, 44 города со справочником часовых поясов, время показывается по местному времени площадки
Интерфейс	Светлая и тёмная темы, адаптивная вёрстка с нижней навигацией на телефоне, экспорт события в календарь (`.ics`)

Принципы

- **API-first.** Вся логика и проверка прав — на бэкенде, веб-клиент лишь один из потребителей REST API.
- **Данные вместо кода.** Формат мероприятия задаётся JSON-схемой в таблице `event_templates`: новый вид активности — строка в БД, а не релиз.
- **Объяснимость.** Рекомендация приходит с причинами и вкладом каждого признака.
- **Предсказуемые ошибки.** Единый формат `{"error": {"code", "message"}}` и осмысленные HTTP-статусы.

ПРОДУКТ

Карта страниц

Веб-клиент — одностраничное приложение (React Router). Маршруты заданы в `frontend/src/App.tsx`. Незнакомый путь перенаправляет на главную.

Маршрут	Экран	Доступ	Главное действие
`/`, `/events`	Главная: карта 2GIS и каталог	все	Найти мероприятие, открыть карточку, записаться
`/events/new`	Конструктор мероприятия	вход; организация — только одобренная	Опубликовать мероприятие
`/events/:id`	Карточка мероприятия	все	Записаться или подать заявку; обсудить; показать QR-билет; завершить (организатор)
`/events/:id/manage`	Управление заявками	организатор мероприятия	Принять или отклонить заявку, исключить участника, сканировать билеты на входе
`/my`	Личный кабинет	вход	«Я участвую», «Я организую», «Архив»; показать билет
`/recommended`	Рекомендации «Для вас»	вход	Открыть подходящее мероприятие, «Не интересно», задать интересы
`/profile`	Собственный профиль	вход	Править профиль и фото, интересы, друзья, подписки, достижения
`/profiles/:type/:id`	Публичный профиль пользователя или организации	все (профиль неодобренной организации — владелец и админ)	Добавить в друзья, подписаться, посмотреть мероприятия

Маршрут	Экран	Доступ	Главное действие
`/auth`	Вход и регистрация	все	Войти, зарегистрироваться как пользователь или организация
`/admin`	Панель администратора	`ROLE_ADMIN`	Организации, мероприятия (в том числе помеченные ⚠), комментарии

Сквозные элементы

- **Верхнее меню и нижняя панель на телефоне** — главная, «Для вас», «Создать», «Мои события», профиль.
- **Панель «Демо-режим»** — вход под любым демо-аккаунтом в один клик (настоящий вход по JWT). См. Акторы и демо-аккаунты.
- **Переключатель темы** — светлая или тёмная, выбор запоминается.
- **Уведомление о достижении** — всплывающая карточка, когда пользователь получает новое достижение.

Пустые состояния и ошибки

Каждый список показывает понятное пустое состояние («В архиве пока нет завершённых событий», «Вы пока не подписаны ни на одну организацию»). Ошибки API выводятся текстом из поля `error.message`, при отсутствии связи — «Нет связи с сервером». Без ключа 2GIS вместо карты показывается заглушка, остальной интерфейс работает.

ПРОДУКТ

Акторы и демо-аккаунты

Роли

Роль	Код	Что может
Гость	—	Читать каталог, карточки мероприятий, обсуждения, справочники и публичные профили; входить и регистрироваться
Пользователь	`ROLE_USER`	Всё, что гость; записываться, создавать мероприятия от своего имени, писать в обсуждениях, дружить, подписываться на организации, получать рекомендации и достижения
Организатор (организация)	`ROLE_ORGANIZER`	Публиковать мероприятия от имени организации — только после одобрения администратором; управлять заявками, отмечать участников на входе
Администратор	`ROLE_ADMIN`	Одобрять и отклонять организации, снимать мероприятия с публикации и возвращать их, скрывать комментарии. Мероприятия не создаёт, в дружбе и подписках не участвует

Права на конкретные объекты (организатор мероприятия, владелец профиля, автор сообщения) проверяются в сервисном слое, а не на клиенте.

Демо-аккаунты

В верхней части интерфейса закреплена панель **«Демо-режим»**: жюри входит под любым сидовым аккаунтом в один клик. Это настоящий вход по JWT — все проверки прав выполняет бэкенд.

Пароль всех демо-аккаунтов: `password123`.

Роль	Аккаунт	Что проверять
Пользователь	Иван Иванов — `user@sport.ru`	Автор «Дворового футбола» (участие по заявке), создание события со своим видом спорта
Пользователь-новичок	Ольга Новичкова — `olga@sport.ru`	Подача заявок, отмена записи, статусы «Ждёт подтверждения» и «Участие подтверждено»
Пользователь-профи	Алексей Профи — `pro@sport.ru`	Автор турнира по настольному теннису — решения по заявкам
Пользователь из Москвы	Мария Москвина — `masha@sport.ru`	Рекомендации в другом городе и часовом поясе
Одобренная организация	СК «Енисей» — `enisey@sport.ru`	Публикация без премодерации, блок «Данные организатора»
Организация на модерации	Клуб «Ладья» — `ladya@sport.ru`	Блокировка публикации до одобрения администратором
Администратор	`admin@sport.ru`	Одобрение и отклонение организаций, снятие мероприятий с публикации

Кроме них в базе есть «фоновые» пользователи и организации (Москва, Новосибирск, Санкт-Петербург, Красноярск): они дают данные для рекомендаций и популярности. Свой аккаунт можно создать на странице `/auth`.

Регистрация

- **Пользователь:** имя, фамилия, e-mail, пароль (8–72 символа). Доступ сразу.
- **Организация:** название, ИНН (10 или 12 цифр), ОГРН (13 или 15), КПП (9), e-mail, пароль. Организация создаётся в статусе `PENDING`; публиковать можно только после одобрения. Отказ приходит с причиной. ИНН и ОГРН уникальны.

ПРОДУКТ

Use cases

Сценарии использования сгруппированы по ролям. Идентификаторы используются в покрытии Т3 и user flows.

Роль	Страница	Идентификаторы
Пользователь	Пользователь	UC-U-01 … UC-U-12
Организатор	Организатор	UC-O-01 … UC-O-09

Роль	Страница	Идентификаторы
Администратор	Администратор	UC-A-01 … UC-A-06

Общие правила

- Гость читает каталог, карточки, обсуждения и профили; для любого действия, меняющего данные, нужен вход.
- Администратор не создаёт мероприятия, не записывается на них, не дружит и не подписывается: бэкенд отвечает `403`.
- Любая ошибка возвращается в формате `{"error": {"code", "message"}}` с осмысленным HTTP-статусом (`400`, `401`, `403`, `404`, `409`).

ПРОДУКТ

Use cases пользователя

ID	Сценарий	Предусловие	Результат
UC-U-01	Найти мероприятие на карте и в каталоге	—	Список и метки для видимой области карты; фильтры: вид спорта (несколько), уровень, режим записи, время, расстояние, «только бесплатные»
UC-U-02	Посмотреть карточку мероприятия	—	Время по местному часовому поясу площадки, место на мини-карте, условия, организатор, обсуждение
UC-U-03	Записаться на мероприятие со свободным входом	Вход; есть свободные места	Участие подтверждено сразу, выдан QR-билет, счётчик мест вырос
UC-U-04	Подать заявку на мероприятие по заявке	Вход; регистрация открыта	Заявка `PENDING` с анкетой и сообщением; место не занято до решения организатора
UC-U-05	Отозвать заявку или отменить участие	Есть заявка или участие	Статус `CANCELLED`, место освобождено; можно записаться снова
UC-U-06	Показать QR-билет на входе	Участие подтверждено	Модальное окно с QR-кодом и кодом билета вида `SPT-XXXXXXXX`
UC-U-07	Задать вопрос организатору в обсуждении	Вход	Сообщение появляется в ленте; ответ организатора помечен значком «Организатор»
UC-U-08	Создать мероприятие	Вход	Мероприятие проходит автопроверку и публикуется от имени пользователя
UC-U-09	Указать интересы и получить рекомендации	Вход	Страница «Для вас» с причинами; «Не интересно» скрывает событие и понижает похожие
UC-U-10	Дружить	Вход	Заявка в друзья (в том числе поиск по e-mail), принятие или отклонение; на карточке мероприятия видно «Идут друзья»
UC-U-11	Подписаться на организацию	Вход; организация одобрена	Подписка в профиле; на собственную организацию подписаться нельзя

ID	Сценарий	Предусловие	Результат
UC-U-12	Получать достижения	Вход	Прогресс и карточка достижений в профиле; уведомление о новом достижении

Правила участия

- Режим `OPEN` — запись подтверждается сразу; режим `APPROVAL` — решает организатор.
- Нельзя записаться на собственное мероприятие, снятое с публикации, завершённое, с закрытой регистрацией, повторно при активной заявке.
- Если в анкете есть обязательные вопросы, их нужно заполнить; если организатор задал требования (справка, страховка, документы) — подтвердить ознакомление.
- Последнее место нельзя выдать дважды: операции с местами выполняются под блокировкой строки мероприятия.

ПРОДУКТ



Use cases организатора

Организатор — автор мероприятия (пользователь) либо владелец организации. Права проверяются по принадлежности мероприятия.

ID	Сценарий	Предусловие	Результат
UC-O-01	Зарегистрировать организацию	—	Организация со статусом `PENDING`; публикация заблокирована до решения администратора
UC-O-02	Собрать мероприятие в конструкторе	Вход; для организации — статус `APPROVED`	Выбран шаблон, заполнены поля, вид спорта (в том числе свой), параметры, анкета участника, место и время
UC-O-03	Указать сведения организатора	Мероприятие от организации	Контакты (обязательны), площадка, справка, страховка, документы, призы, программа, условия отмены, партнёры
UC-O-04	Принять или отклонить заявку	Есть заявки `PENDING`	`CONFIRMED` (выдаётся билет) или `REJECTED` с комментарием; участник видит новый статус
UC-O-05	Исключить подтверждённого участника	Участие `CONFIRMED`	Статус `REJECTED`, место освобождено
UC-O-06	Ответить в обсуждении	Вход	Ответ помечен значком «Организатор» (ставит бэкенд)
UC-O-07	Отметить участника на входе	Мероприятие организатора	Сканирование QR камерой или ввод кода вручную; статусы `SUCCESS`, `ALREADY_CHECKED_IN`, `NOT_FOUND`, `WRONG_EVENT`, `NOT_CONFIRMED`
UC-O-08	Завершить мероприятие	Время окончания прошло; статус `PUBLISHED`	Мероприятие переходит в `ARCHIVED` и попадает в архив организатора и участников
UC-O-09	Исправить отклонённое автопроверкой	Мероприятие `UNPUBLISHED` с причиной	Автор видит причину на форме и в «Моих событиях»; администратор может вернуть мероприятие

Статусы организации

`PENDING` → `APPROVED` (публикация без премодерации) либо `REJECTED` (с причиной, видна владельцу). Профиль неодобренной организации виден только владельцу и администратору.

ПРОДУКТ

Use cases администратора

Панель администратора — `/admin`. Все методы `/api/v1/admin/**` доступны только `ROLE_ADMIN`.

ID	Сценарий	Результат
UC-A-01	Одобрить организацию	Статус `APPROVED`; организация публикует мероприятия сразу, без премодерации
UC-A-02	Отклонить организацию	Статус `REJECTED` с обязательной причиной, которую видит владелец
UC-A-03	Снять мероприятие с публикации	Статус `UNPUBLISHED` с причиной; исчезает из каталога, карты и рекомендаций; по прямой ссылке видна плашка, запись закрыта
UC-A-04	Вернуть мероприятие	Статус `PUBLISHED`, пометка автопроверки снимается; так исправляется ошибочное отклонение
UC-A-05	Разобрать помеченные автопроверкой мероприятия	Список сортируется так, что ⚠ идут первыми; видны объяснение ИИ, найденные слова и модель; кнопка «Проверено» снимает пометку
UC-A-06	Скрыть комментарий	Мягкое скрытие: обычные пользователи видят «Сообщение скрыто модератором», настоящий текст виден только администратору; вкладка «Комментарии» показывает ленту с названиями мероприятий

Фильтры панели

Мероприятия: «Все», «Требуют внимания» (помечены ⚠), «Отклонены автопроверкой». У вкладки организаций счётчик заявок, ожидающих решения (`PENDING`), показан на самой вкладке.

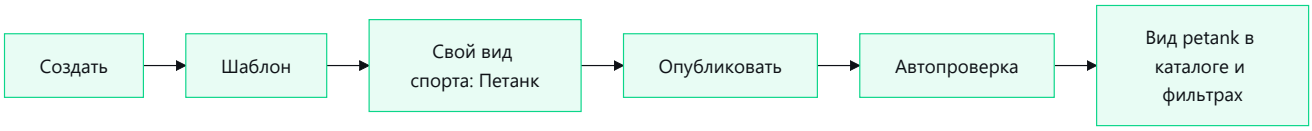
ПРОДУКТ

User flows

Пошаговые пути по ключевым сценариям. Первые шесть — обязательные сценарии ТЗ; проверять их нужно на свежей базе (`docker compose down -v && docker compose up --build -d`).

1. Создание мероприятия с новым видом спорта

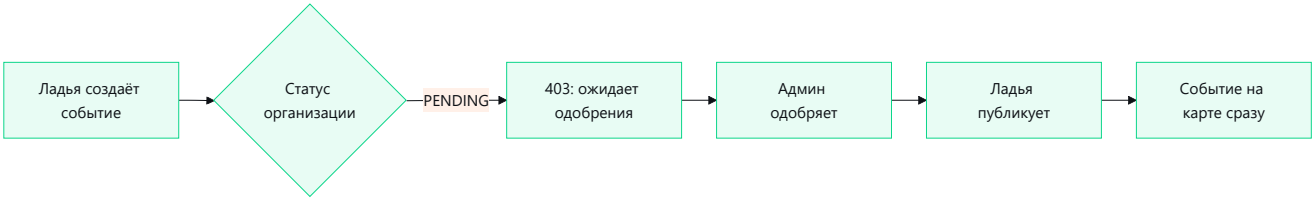
Кто: Иван (`user@sport.ru`).



Название нормализуется в slug (`Воркаут` → `vorkaut`), вид попадает в общий каталог, фильтры и `GET /sports`. Мероприятие со своим видом спорта всегда проходит проверку ИИ.

2. Допуск организации и публикация

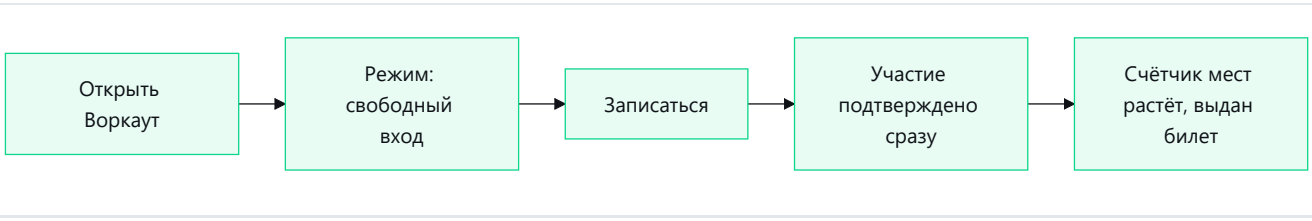
Кто: Клуб «Ладья» (`ladya@sport.ru`) и администратор (`admin@sport.ru`).



Новую организацию можно зарегистрировать на `/auth`: название, ИНН, ОГРН, КПП.

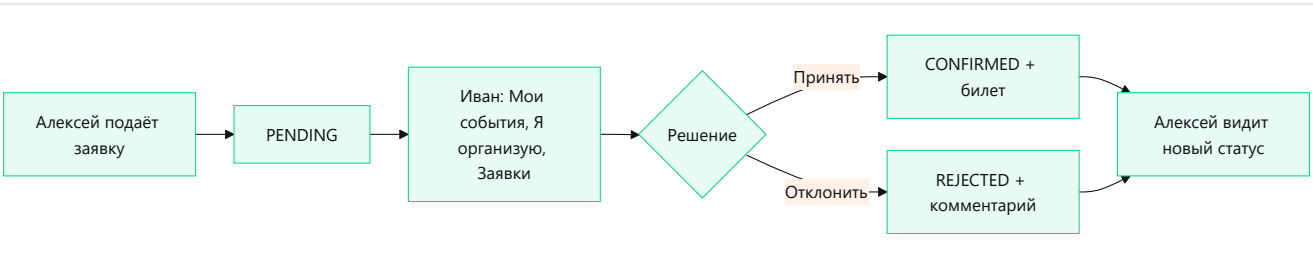
3. Свободное присоединение

Кто: Ольга (`olga@sport.ru`).



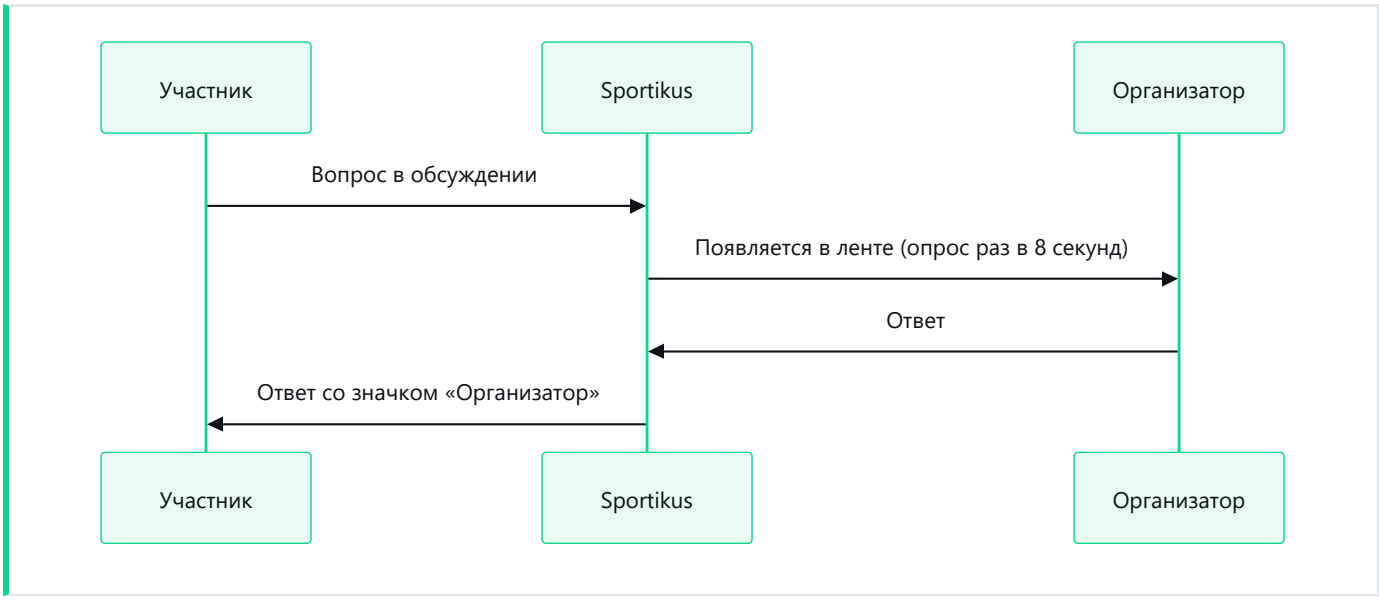
4. Участие по заявке

Кто: Иван (организатор) и Алексей (`pro@sport.ru`, участник).



5. Уточнение условий в обсуждении

Кто: Ольга или Алексей и Иван (автор).



Администратор может скрыть сообщение — остальные увидят «Сообщение скрыто модератором».

6. Снятие мероприятия с публикации

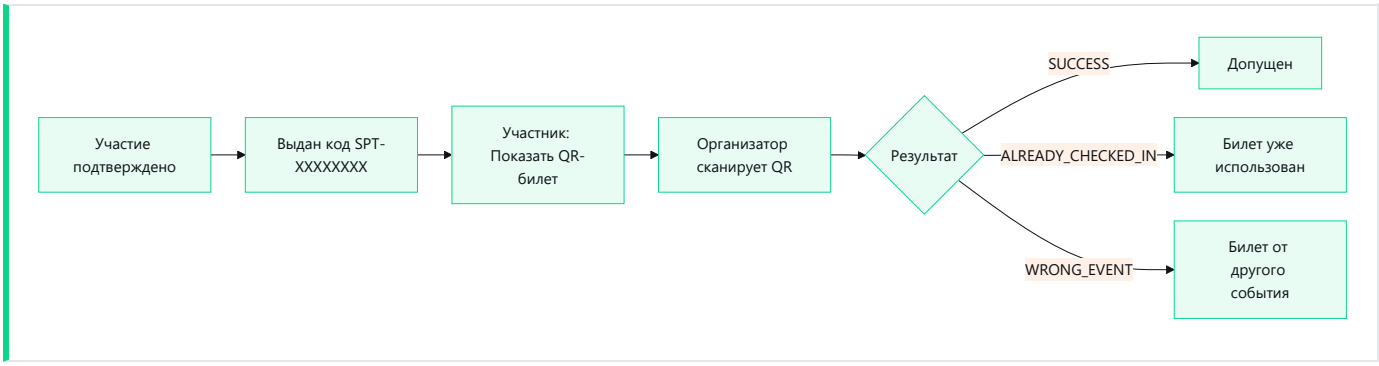
Кто: Администратор.



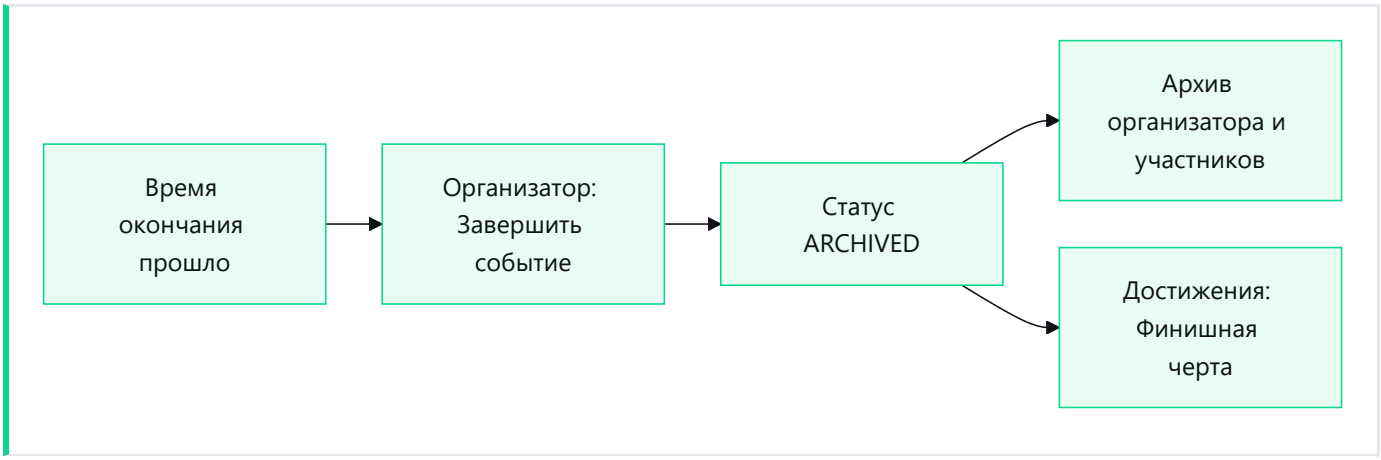
Пример: `/events/5`.

7. Билет и вход на мероприятие

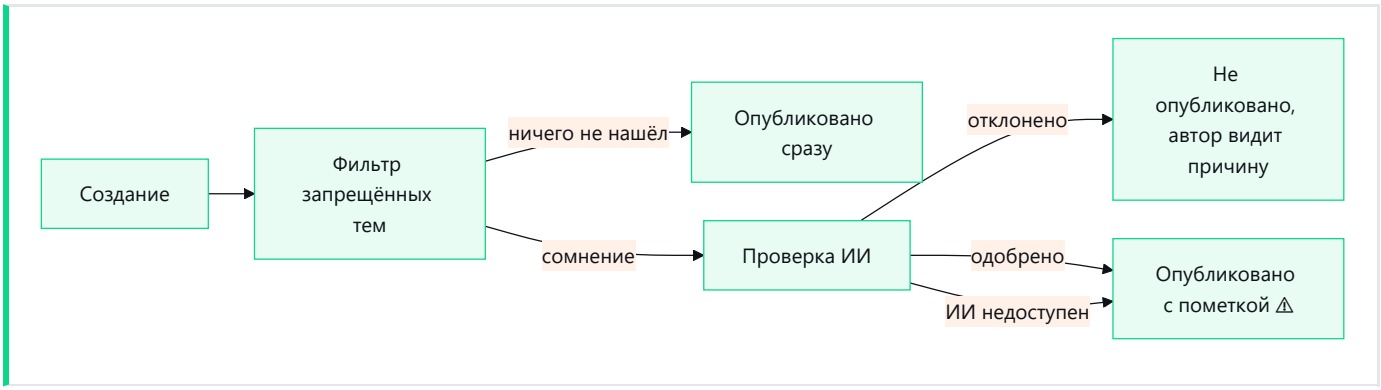
Кто: участник и организатор.



8. Завершение и архив



9. Автопроверка нового мероприятия



Подробности — в разделе Автопроверка мероприятий.

Покрытие ТЗ

Документ сопоставляет требования ТЗ с реализацией. Для шести обязательных сценариев «выполнено» означает, что сценарий проходит на свежей базе при ручной проверке. Для остальных возможностей «реализовано» описывает то, что есть в коде и API; тестами они покрыты в разной степени (см. Тестирование).

Шесть обязательных сценариев

Все проверки — на свежей базе: `docker compose down -v && docker compose up --build -d`.

№	Сценарий	Статус	Как проверить
1	Создание мероприятия с новым видом спорта	Выполнено	Войти как Иван, «Создать», «+ Свой вид спорта», ввести «Петанк». Вид получает slug `petank`, появляется в фильтрах и `GET /sports`
2	Допуск организации и публикация	Выполнено	«Ладья» пытается создать событие — блокировка (`403`). Администратор одобряет организацию, «Ладья» публикует сразу, без премодерации
3	Свободное присоединение	Выполнено	Ольга открывает «Воркаут: тренировка и разбор базы» (свободный вход) и записывается — «Участие подтверждено» сразу
4	Участие по заявке	Выполнено	У «Дворового футбола 5x5» есть заявка Алексея. Иван принимает или отклоняет её в «Мои события — Я организую — Заявки»; Алексей видит новый статус
5	Уточнение условий (обсуждение под событием)	Выполнено	Ольга или Алексей пишет вопрос под «Дворовым футболом 5x5», Иван отвечает — ответ со значком «Организатор». Есть демо-диалоги. Администратор скрывает сообщение
6	Снятие мероприятия с публикации	Выполнено	Администратор: «Мероприятия — Снять с публикации» с причиной. Событие исчезает из каталога, карты и рекомендаций; по прямой ссылке — плашка с причиной, запись закрыта

Сверх обязательного

Возможность	Статус	Комментарий
Конструктор мероприятий на данных (JSON-схема шаблонов)	Реализовано	Новый формат — строка в `event_templates`, без правок кода. См. Конструктор
Рекомендации на масштаб страны	Реализовано	PostGIS, четыре источника кандидатов, объяснимое ранжирование, сигналы с затуханием. См. Рекомендательная система
Автопроверка мероприятий (фильтр + ИИ)	Реализовано	Покрыта тестами; ИИ вызывается вне транзакции, при недоступности автор не блокируется. См. Автопроверка
Электронные QR-билеты и отметка на входе	Реализовано	Код `SPT-XXXXXXXX`, пять исходов проверки
Друзья, подписки на организации, достижения	Реализовано	Видно «Идут друзья» на карточке
Архив завершённых мероприятий	Реализовано	Завершает только организатор и только после времени окончания

Возможность	Статус	Комментарий
Кэш карты в Redis	Реализовано	Необязателен: без Redis запросы идут в БД
Тёмная тема, адаптивная вёрстка, экспорт `.ics`	Реализовано	—

Что не реализовано

Честный список — в Ограничениях и развитии. Главное:

- PWA (манифест, офлайн) и push-уведомления.
- Уведомления об ответах в обсуждении; обсуждение обновляется опросом раз в 8 секунд, без WebSocket.
- Сбор интересов при регистрации: интересы указываются в профиле или на странице рекомендаций.
- Рекомендации — взвешенная формула, а не обученная модель: реальных данных для обучения пока нет.

Известные риски для демонстрации

- Ключ 2GIS встраивается при сборке фронтенда; без него вместо карты показывается заглушка.
- Без ключа RouterAI автопроверка работает только на словаре, а сомнительные мероприятия публикуются с пометкой ⚠.
- Демо-данные создаются миграциями; тестовые мероприятия, созданные вручную во время проверки, остаются в базе до её пересоздания.

БИЗНЕС

Нефункциональные требования

Таблица показывает, как каждое требование выполнено в текущей версии и где проходит граница.

Область	Требование	Как выполнено
Масштаб	Работа по всей стране, а не в одном городе	Геопоиск на PostGIS с GiST-индексом по опубликованным событиям; каталог ограничен областью (центр + радиус до 300 км) и 500 событиями; клиент запрашивает видимую область карты
Время	Корректное время в любом часовом поясе	Время хранится в UTC; клиент показывает по часовому поясу площадки и подписывает «местное»; ввод при создании трактуется как местное время площадки; 44 города со справочником поясов
Производительность	Быстрый ответ карты и рекомендаций	Кэш списка мероприятий для карты в Redis (TTL 60 с, версия данных `map:version`); кэш рекомендаций на пользователя на 2 минуты; ограниченные выборки кандидатов (300 на источник); тяжёлые расчёты в фоне
Консистентность	Нельзя выдать последнее место дважды	Все операции с местами — под `SELECT … FOR UPDATE` на строке мероприятия
Безопасность	Аутентификация и авторизация	JWT (срок 24 часа), без сессий, пароли BCrypt; `/api/v1/admin/**` — только `ROLE_ADMIN`; права на объекты проверяются в сервисах

Область	Требование	Как выполнено
Безопасность	Безопасная загрузка файлов	Только JPG, PNG, WebP до 5 МБ; имя файла — случайный UUID; отдельный каталог и volume
Безопасность	Защита от злоупотреблений	Автопроверка новых мероприятий; уникальность ИНН и ОГРН; текст мероприятия передаётся ИИ как данные (защита от prompt injection)
Отказоустойчивость	Сбой внешних сервисов не ломает продукт	Недоступен Redis — запросы идут в БД; недоступен ИИ — мероприятие публикуется с пометкой ⚠; нет ключа 2GIS — заглушка вместо карты
Предсказуемость	Единый формат ошибок	`{"error": {"code", "message"}}`, статусы `400/401/403/404/409`
Наблюдаемость	Состояние системы	`/actuator/health`, `/actuator/info`, `/actuator/metrics`; Swagger UI и OpenAPI JSON
Переносимость	Запуск одной командой	`docker compose up --build -d`: PostgreSQL + PostGIS, Redis, backend, frontend (Nginx)
Сопровождаемость	Изменение схемы без потери данных	Flyway-миграции V1–V8 и повторяемая `R__demo_data.sql`; Hibernate в режиме `validate`
Масштабируемость	Работа нескольких экземпляров бэкенда	Фоновые задачи берут `pg_try_advisory_xact_lock` и выполняются ровно в одном экземпляре
Качество	Автоматические проверки	Юнит-тесты бэкенда (Maven) и фронтенда (Vitest). См. Тестирование
Интерфейс	Адаптивность и темы	Светлая и тёмная темы, нижняя навигация на телефоне

Известные упрощения

CORS открыт для всех источников; загрузки лежат на диске; кэш рекомендаций локален процессу; обсуждение обновляется опросом. Что менять при росте нагрузки — в Ограничениях и развитии.

Инженерия

Раздел описывает текущую реализацию монорепозитория. Поведение продукта для пользователя описано в разделах «Продукт» и «Бизнес»; здесь — как это устроено внутри.

Страница	О чём
Архитектура	Принципы, компоненты, модули backend, ключевые потоки, безопасность, архитектурные решения
Модель данных	ER-диаграмма, ограничения целостности, миграции V1–V8
API	Группы эндпоинтов, доступ, формат ошибок
Конструктор мероприятий	Шаблоны, поля, анкета участника, валидатор
Рекомендательная система	Кандидаты, ранжирование, сигналы, кэш, фоновые задачи
Автопроверка мероприятий	Фильтр запрещённых тем и проверка ИИ
Локальный запуск	Docker, разработка без Docker, переменные окружения
Тестирование	Что покрыто тестами бэкенда и фронтенда
Ограничения и развитие	Что не реализовано и что менять при росте нагрузки

Стек

Слой	Технологии
Backend	Java 21, Spring Boot 3.3, Spring Data JPA, Spring Security (JWT), Flyway, Springdoc OpenAPI
База данных	PostgreSQL 16 + PostGIS (геопоиск, материализованные представления, секционированный журнал)
Кэш	Redis 7 (кэш карты; необязателен)
Frontend	React 19, Vite, TypeScript, Tailwind CSS 4, React Router, 2GIS MapGL JS API, Lucide Icons, `qrcode` и `html5-qrcode` для билетов
Тесты	JUnit 5 + AssertJ (backend), Vitest + Testing Library (frontend)
DevOps	Docker, Docker Compose, Nginx

Структура репозитория

```
.
├── backend/                    # Spring Boot (package-by-feature, ru.sportikus)
│   └── src/main
│       ├── java/ru/sportikus/
│       │   ├── auth, security   # вход, регистрация, JWT
│       │   ├── user, organization, sport, city
│       │   ├── template, form   # конструктор: шаблоны, поля, валидатор
│       │   ├── event, participation, comment, admin
│       │   ├── friendship, subscription, achievement
│       │   ├── moderation       # автопроверка: фильтр + ИИ (RouterAI)
│       │   ├── preference, recommendation
│       │   └── common, config   # ошибки, Security, OpenAPI, планировщик
│       └── resources/db/migration   # V1–V8 + R__demo_data
├── frontend/                   # React + Vite + TypeScript
├── docs/architecture.md        # исходный технический обзор
├── docs-zensical/              # эта документация (Zensical)
├── scripts/build_pdf.py        # сборка PDF из этой документации
└── docker-compose.yml          # postgres (PostGIS), redis, backend, frontend
```

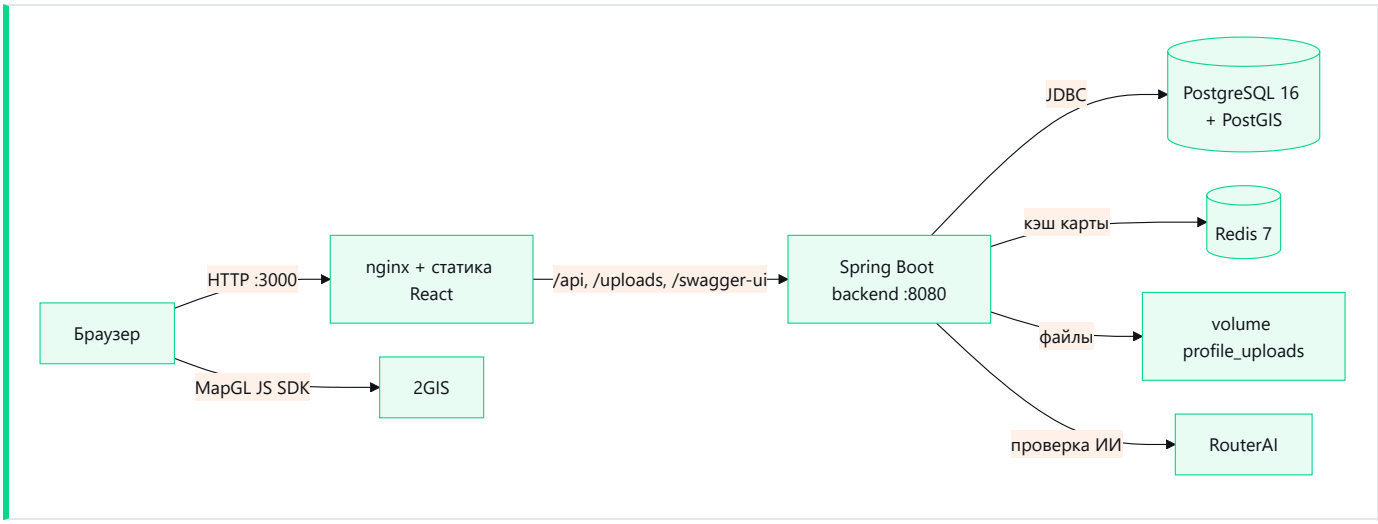
ИНЖЕНЕРИЯ

Архитектура

Принципы

- **API-first.** Вся бизнес-логика и проверка прав — на Java-бэкенде; веб-клиент — один из потребителей REST API. Тот же API подходит мобильному приложению.
- **Данные вместо кода.** Формат мероприятия (поля, вопросы участнику, значения по умолчанию) задаётся JSON-схемой в таблице `event_templates`.
- **Package-by-feature.** Каждый модуль (`event`, `participation`, `recommendation`…) содержит свои `controller / service / repository / model / dto`; связи между модулями идут через сервисы и доменные события.
- **Масштаб страны с первого дня.** Геопоиск на PostGIS с GiST-индексом, местное время площадки, ограниченные выборки, расчёты в фоне.
- **Объяснимость.** Рекомендация приходит с причинами и вкладом каждого признака.
- **Предсказуемые ошибки.** Единый формат `{"error": {"code", "message"}}`.

Компоненты и развёртывание



Четыре контейнера в `docker-compose.yml`:

Сервис	Образ	Роль
`postgres`	`postgis/postgis:16-3.4-alpine`	Данные, геоиндекс, материализованные представления
`redis`	`redis:7-alpine`	Только кэш: без сохранения на диск, память 128 МБ, вытеснение `allkeys-lru`
`backend`	сборка `backend/Dockerfile` (Maven → JRE 21)	REST API, JWT, фоновые задачи, раздача `/uploads/**`
`frontend`	сборка `frontend/Dockerfile` (Node → nginx)	SPA и reverse-proxy к бэкенду

Миграции схемы выполняет Flyway при старте бэкенда. Hibernate работает в режиме `ddl-auto: validate` и схему не меняет. Включён `out-of-order`, чтобы миграция, пришедшая из слияния ветки позже уже применённой, выполнялась на существующих базах; на чистой базе порядок обычный.

Модули backend

Пакет `ru.sportikus`:

Модуль	Ответственность
`auth`, `security`	Вход, регистрация пользователя и организации, JWT (`JwtService`, `JwtAuthFilter`), `CurrentUser`
`user`	Пользователи, публичные профили, загрузка фото, личный кабинет (`/me/*`)
`organization`	Организации и статусы `PENDING → APPROVED / REJECTED`, реквизиты ИНН, ОГРН, КПП
`sport`	Справочник видов спорта; новый вид нормализуется в slug (`SlugUtil`)
`template`, `form`	Конструктор: шаблоны, описание полей (`FieldDef`), универсальный валидатор (`FieldValidator`)
`event`	Мероприятия, каталог с фильтрами и геопоиском, завершение и архив, экспорт `.ics`, кэш карты (`event.cache`)
`participation`	Запись и заявки, решения организатора, блокировка мест, билеты и отметка на входе

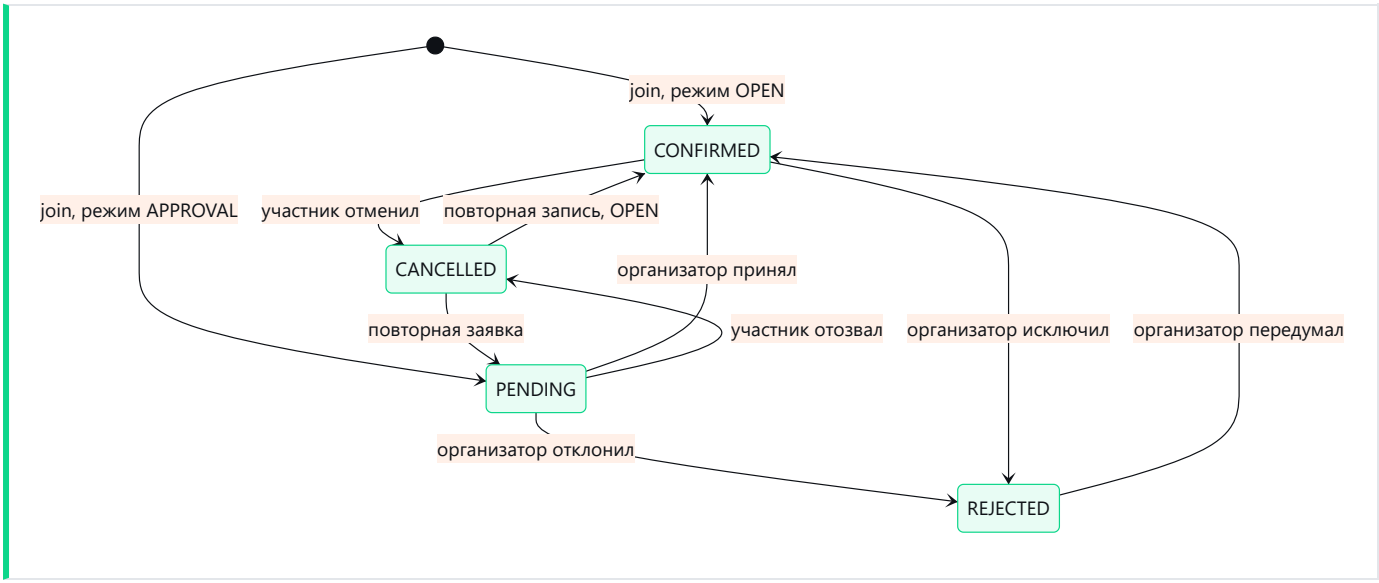
Модуль	Ответственность
`comment`	Обсуждение: сообщения, значок организатора, мягкое скрытие
`friendship`	Заявки в друзья, принятие, поиск по e-mail, «Идут друзья» на карточке
`subscription`	Подписки пользователей на одобренные организации
`achievement`	30 достижений по метрикам активности, считаются запросом при обращении
`moderation`	Автопроверка мероприятий: фильтр запрещённых тем, проверка ИИ через RouterAI
`admin`	Модерация организаций и снятие мероприятий с публикации
`city`	Справочник городов, ближайший город к точке, часовые пояса
`preference`	Интересы и предпочтения пользователя
`recommendation`	Источники кандидатов, ранжирование, сигналы, кэш, журнал, фоновые задачи
`common`, `config`	`ApiException`, обработчик ошибок, Security, OpenAPI, планировщик, раздача файлов

Ключевые потоки

Публикация мероприятия

1. Клиент загружает шаблоны (`GET /templates`), пользователь выбирает формат и заполняет поля.
2. `POST /events`: проверяется роль (администратор не создаёт), для организации — статус `APPROVED` (иначе `403`) и обязательные контакты; значения полей валидируются по схеме шаблона, вопросы участнику — по определениям.
3. Вид спорта находится по id или создаётся по названию (slug), по координатам определяются ближайший город и часовой пояс, поля шаблона копируются в событие снимком.
4. Автопроверка: чистое мероприятие публикуется сразу, сомнительное — после решения ИИ. Ручной премодерации нет. См. Автопроверка.
5. После фиксации транзакции сбрасывается кэш карты.

Участие с подтверждением



Все операции с местами выполняются под `SELECT … FOR UPDATE` на строке мероприятия: последнее место нельзя выдать дважды. Проверяются собственное мероприятие, снятие с публикации, завершение, закрытая регистрация, повторные заявки, лимит мест, обязательные ответы анкеты и подтверждение требований. Каждое изменение статуса публикует `ParticipationChangedEvent`, на который подписана рекомендательная система.

Билеты и отметка на входе

Когда участие становится `CONFIRMED` (сразу при `OPEN` или после решения организатора), сервис выдаёт код билета `SPT-` и восемь символов случайного UUID в верхнем регистре. Код уникален (`UNIQUE` в `participations.ticket_code`). Клиент строит QR-код библиотекой `qrcode`.

`POST /events/{eventId}/checkin` с телом `{"ticketCode": "…"}` доступен организатору мероприятия и администратору. Код приводится к верхнему регистру и проверяется последовательно:

Статус	Когда
`NOT_FOUND`	Код пустой или билета нет в системе
`WRONG_EVENT`	Билет принадлежит другому мероприятию
`NOT_CONFIRMED`	Участие не в статусе `CONFIRMED`
`ALREADY_CHECKED_IN`	Билет уже использован (возвращается время первой отметки)
`SUCCESS`	Участник допущен: сохраняются `checked_in`, время и кто отметил

Ответ содержит участника, название мероприятия, признак платного входа и цену — организатор видит, нужно ли брать оплату. Сканер камеры клиента построен на `html5-qrcode`, есть ввод кода вручную.

Завершение и архив

`POST /events/{id}/complete` переводит мероприятие в `ARCHIVED`. Условия: вызывает организатор, время окончания прошло, статус `PUBLISHED`; повторное завершение и завершение снятого или отменённого мероприятия отвечают `409`. Архив организатора и подтверждённых участников показывается в «Моих событиях». Каталог и карта показывают только опубликованные мероприятия, поэтому архивные из них исчезают.

Обсуждение под мероприятием

Таблица `comments` (`text`, `is_organizer`, `is_hidden`). Читать обсуждение можно без входа, писать — после входа (до 2000 символов). Признак `is_organizer` выставляет сервис: автор мероприятия или владелец организации получают значок «Организатор». Скрытие мягкое: `CommentDto` подменяет текст заглушкой «Сообщение скрыто модератором» для всех, кроме администратора. Скрыть может администратор, автор сообщения и организатор мероприятия. Клиент обновляет обсуждение опросом раз в 8 секунд.

Дружба, подписки, достижения

- **Дружба.** Одна строка в `friendships` на пару пользователей независимо от того, кто отправил заявку (уникальный индекс по `LEAST/GREATEST`). Статусы `PENDING` и `ACCEPTED`. Нельзя дружить с собой и с администратором; встречная заявка требует принять или отклонить входящую. Поиск — по точному e-mail, без администраторов. «Идут друзья» подмешивается в ответ для каждого просматривающего пользователя отдельно, поэтому в кэш карты не попадает.

- **Подписки.** Подписаться можно только на одобренную организацию, не на собственную; повторная подписка ничего не меняет.
- **Достижения.** Метрики считаются SQL-запросами по участиям, дружбе, созданным и архивным мероприятиям, видам спорта. Достижение «получено» при `прогресс ≥ цель`; три мета-достижения считают уже полученные.

Модерация

Администратор одобряет или отклоняет организацию (причина обязательна), снимает мероприятия с публикации (причина обязательна) и скрывает комментарии. Снятое мероприятие исчезает из каталога, карты и рекомендаций, но доступно по прямой ссылке с плашкой причины; запись закрыта.

Кэш карты

Список мероприятий для карты не зависит от пользователя, поэтому лежит в Redis (`MapCache`, TTL 60 секунд).

- **Ключ.** Фильтр нормализуется: центр области округляется до 0,01° (около 1 км), радиус увеличивается на запас и поднимается до ступени 1, 3, 5, 10, 25, 50, 100, 300 км, чтобы область всегда накрывала запрошенную. Запросы с текстовым поиском в кэш не кладутся.
- **Сброс.** Версия данных `map:version` увеличивается при создании мероприятия, снятии с публикации и возврате, завершении и после решения автопроверки. Версию читают до запроса в БД: если данные изменились, пока шёл запрос, устаревший результат не записывается под новую версию.
- **Личное.** Друзья, своя заявка и признак «можно управлять» подмешиваются на каждый запрос (`EventMapper.withViewer`), в кэше их нет.
- **Свежесть счётчиков.** Запись и отмена участия кэш не сбрасывают, поэтому число подтверждённых мест в списке карты может отставать до 60 секунд (TTL). Карточка мероприятия кэш не использует и всегда показывает точные значения.
- **Отказоустойчивость.** Любая ошибка хранилища — промах кэша, а не ошибка запроса. Без Redis (или при `MAP_CACHE_ENABLED=false`) подставляется `NoOpMapCache`.

География и время

- Координаты хранятся в `lat` и `lng`; вычисляемая колонка `geog` типа `geography(Point, 4326)` индексируется GiST (частичный индекс только по опубликованным событиям).
- Каталог всегда ограничен областью (центр + `radiusKm`, не больше 300 км) и 500 событиями.
- Справочник `cities` содержит 44 города с часовыми поясами. Ближайший город (KNN по `geog <-> точка`) определяет `city_id` и `time_zone`.
- Время передаётся в UTC. Клиент показывает его в часовом поясе площадки и подписывает «местное», если оно отличается от пояса устройства.

Безопасность

- **Аутентификация:** JWT (подпись `JWT_SECRET`, срок 24 часа), без сессий; пароли — BCrypt.
- **Авторизация:** публичны только чтение каталога, карточки события, справочников, обсуждения, профилей и достижений профиля, вход и регистрация; `/api/v1/admin/**` — только `ROLE_ADMIN`; остальное требует токен. Права на объекты проверяются в сервисах.
- **Организации:** уникальные ИНН и ОГРН, проверка длины реквизитов, публикация только после одобрения.
- **Файлы:** JPG, PNG, WebP до 5 МБ, имя — случайный UUID, отдельный volume.
- **Ошибки:** внутренние детали не раскрываются, клиент получает типизированный код и сообщение.

Архитектурные решения

Решение	Почему
Рекомендации на PostgreSQL + PostGIS, без Elasticsearch	Важны свежие данные (места, статус, заявки) и геопоиск, а не полнотекстовый поиск; отдельный индекс отставал бы от БД и дублировал данные
Формат мероприятия — JSON-схема в БД	Платформа гибка к любым активностям; снимок схемы в событии не ломается при правке шаблона
Взвешенная формула вместо обученной модели (LTR)	Реальных данных для обучения пока нет, а модель на сгенерированных данных лишь повторила бы формулу. Журнал `recommendation_log` и интерфейс `Ranker` готовы к замене
Кэш рекомендаций в памяти, кэш карты в Redis	Рекомендации персональны, живут в памяти экземпляра (`RecommendationCache` позволяет перейти на Redis); карта общая для всех, поэтому в Redis
Блокировка строки мероприятия	Простая и надёжная защита от двойной выдачи места; нагрузка на одно мероприятие невелика
Фоновые задачи с advisory-блокировкой	Не нужен внешний планировщик, при масштабировании задача не выполняется дважды
Доменные события участия и интересов	Рекомендации обновляются и сбрасывают кэш без жёсткой связи модулей
Автопроверка: словарь, затем ИИ только при сомнении	Словарь мгновенный и бесплатный, но не понимает контекст; ИИ понимает контекст, но стоит денег и секунд
Быстрая модель через RouterAI	`gemini-2.5-flash-lite` отвечает за 1–3 секунды и стоит доли копейки за проверку; шлюз OpenAI-совместимый, модель меняется настройкой
При недоступности ИИ публиковать с пометкой	Добросовестные авторы не ждут из-за сбоя внешнего сервиса; риск закрывает ручная проверка
2GIS MapGL на клиенте	Подробная карта российских городов; без ключа приложение работает с заглушкой

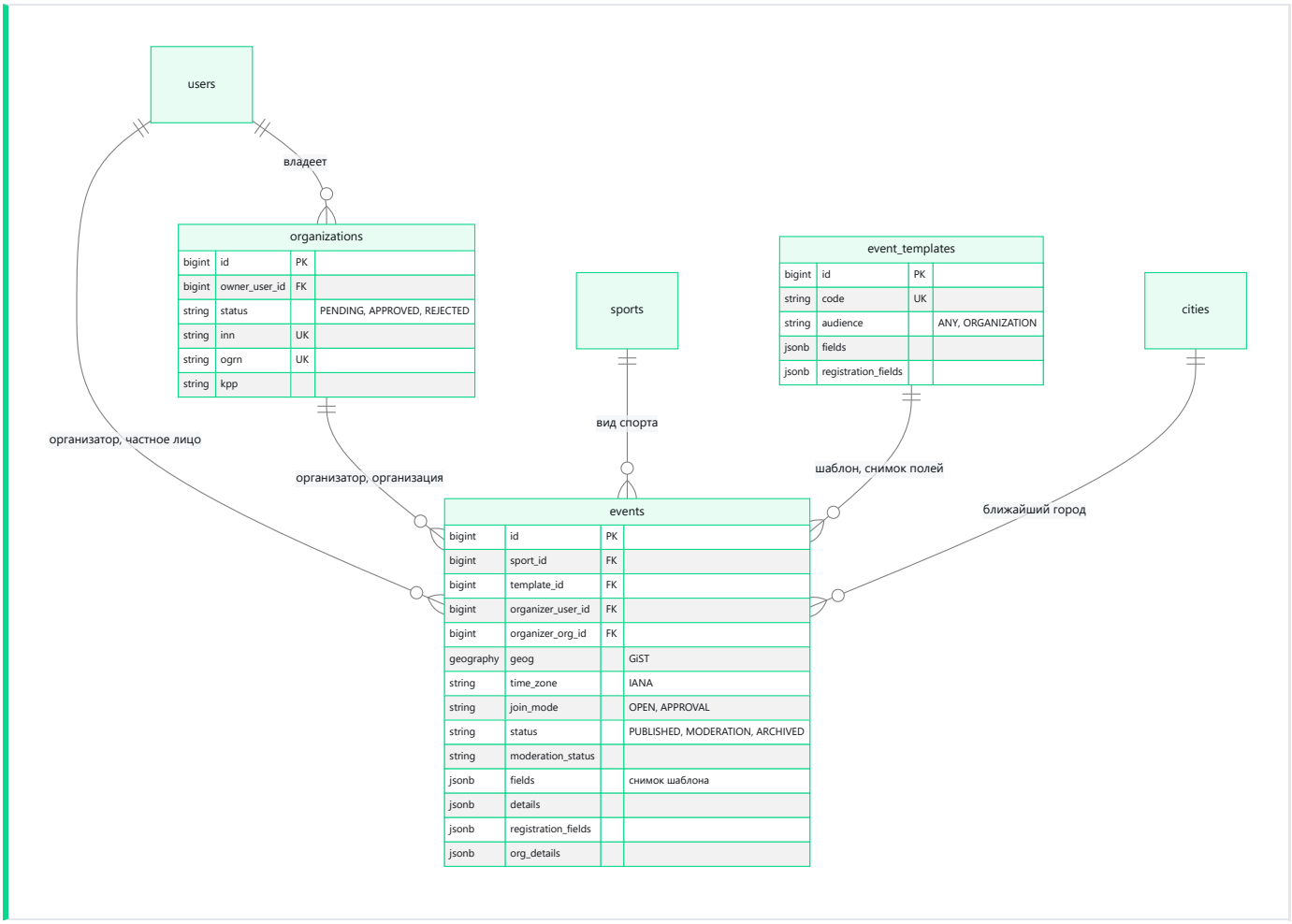
ИНЖЕНЕРИЯ



Модель данных

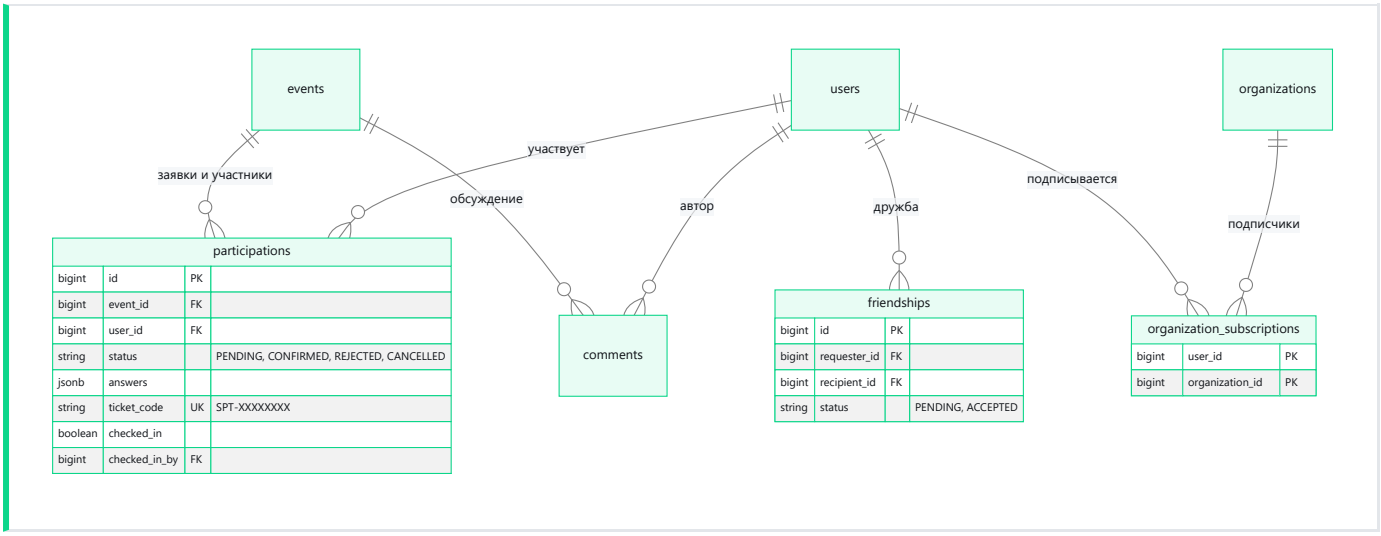
PostgreSQL 16 с расширением PostGIS. Схему создают и меняют только Flyway-миграции.

Мероприятия и организации

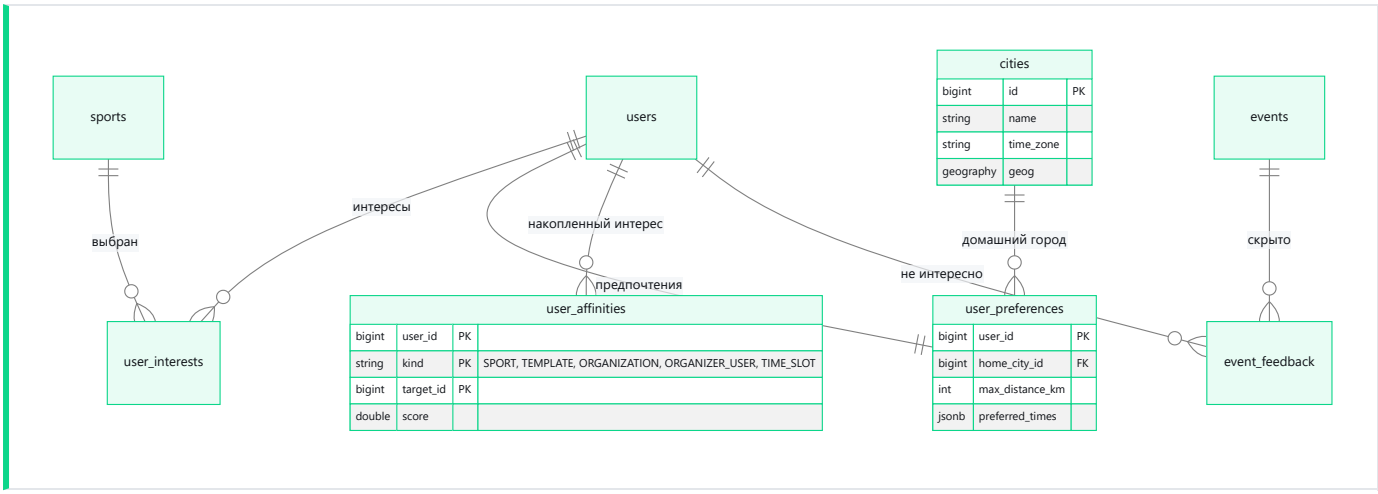


У события ровно один организатор: либо `organizer_user_id`, либо `organizer_org_id`. Статусы события — `PUBLISHED`, `UNPUBLISHED`, `CANCELLED`, `MODERATION`, `ARCHIVED`; статусы автопроверки — `CLEAN`, `PENDING`, `FLAGGED`, `REJECTED`, `REVIEWED`.

Участие, обсуждение и социальные связи



Персонализация и рекомендации



Отдельно, без внешних ключей: `recommendation_log` (журнал показов и действий, секционирован по месяцам) и два материализованных представления — `mv_event_popularity` и `mv_sport_similarity`.

Ограничения целостности

- `CHECK`: у события ровно один организатор; `end_time > start_time`; `capacity > 0`.
- Уникальность `(event_id, user_id)` в участиях и `ticket_code` у билетов.
- Частичные уникальные индексы по ИНН и ОГРН организаций.
- В `friendships` — `CHECK (requester_id <> recipient_id)`, статус из набора `PENDING` и `ACCEPTED`, уникальный индекс по неупорядоченной паре `(LEAST, GREATEST)`: одна строка на пару пользователей.
- У подписок составной первичный ключ `(user_id, organization_id)`.

Миграции

Каталог `backend/src/main/resources/db/migration`:

Файл	Содержимое
`V1__init_schema.sql`	Пользователи, организации, виды спорта, мероприятия, участия, комментарии
`V2__event_constructor.sql`	Шаблоны конструктора, детали организаций, поля участия, справочник видов спорта
`V3__recommendations.sql`	PostGIS, города и часовые пояса, интересы, сигналы, обратная связь, лог, материализованные представления
`V4__registration_profiles.sql`	Реквизиты организаций (ИНН, ОГРН, КПП) и их уникальность
`V5__event_moderation.sql`	Результат автопроверки мероприятия: статус, причина, находки фильтра, модель ИИ
`V6__friendships.sql`	Дружба пользователей
`V7__organization_subscriptions.sql`	Подписки на организации
`V8__tickets_and_checkin.sql`	Код билета, отметка о входе; коды для уже подтверждённых участий
`R__demo_data.sql`	Повторяемая миграция: идемпотентные демо-данные

> **Порядок миграций**
>
> Включён `spring.flyway.out-of-order`, потому что `V5` пришла из слияния ветки позже уже применённой `V6`. На чистой базе миграции идут в обычном порядке.

ИНЖЕНЕРИЯ

API

Полная интерактивная документация — **Swagger UI**: `http://localhost:8080/swagger-ui/index.html` (доступна и через порт 3000). Спецификация OpenAPI — `http://localhost:8080/v3/api-docs`.

Все пути имеют префикс `/api/v1`. Загруженные изображения отдаются по `/uploads/{uuid}.{ext}`.

Авторизация

`POST /api/v1/auth/login` возвращает `token`. Дальше его передают в заголовке `Authorization: Bearer <token>`; в Swagger UI — кнопка **Authorize**. Токен действует 24 часа.

Формат ошибок

```
{ "error": { "code": "FORBIDDEN", "message": "Недостаточно прав для этого действия" } }
```

Статус	Когда
`400`	Некорректные данные запроса
`401`	Нет или недействителен токен
`403`	Недостаточно прав (например, организация ещё не одобрена)
`404`	Объект не найден
`409`	Конфликт состояния (нет мест, повторная заявка, завершение до времени окончания)

Эндпоинты

Группа	Методы и пути	Доступ
Auth	`POST /auth/login` · `POST /auth/register/user` · `POST /auth/register/organization`	публично
	`GET /auth/me`	токен
Профили	`GET /profiles/users/{id}` · `GET /profiles/organizations/{id}` · `GET /profiles/users/{id}/achievements`	публично
	`PATCH /profiles/users/{id}` · `PATCH /profiles/organizations/{id}` · `POST …/{id}/image` (multipart)	владелец

Группа	Методы и пути	Доступ
Каталог	`GET /events` (фильтры `sport`, `q`, `skillLevel`, `joinMode`, `template`, `lat`, `lng`, `radiusKm`) · `GET /events/{id}` · `GET /events/{id}/ics`	публично
	`GET /sports` · `GET /templates` · `GET /cities` · `GET /cities/nearest`	публично
	`POST /events`	токен; организация — только одобренная
	`POST /events/{id}/complete` — завершить и перенести в архив	организатор мероприятия
Обсуждение	`GET /events/{id}/comments`	публично
	`POST /events/{id}/comments` · `PATCH /events/{id}/comments/{commentId}/hide`	токен; скрыть — админ, автор или организатор
Участие	`POST /events/{id}/join` · `DELETE /events/{id}/join`	токен
	`GET /events/{id}/participations` · `PATCH /events/{id}/participations/{pid}`	организатор мероприятия
	`POST /events/{id}/checkin` (`{"ticketCode"}`) — отметка на входе	организатор мероприятия, админ
Личный кабинет	`GET /me/participations` · `GET /me/events` · `GET /me/achievements`	токен
	`GET /me/preferences` · `PUT /me/preferences`	токен
Друзья	`POST /friends/{userId}` · `DELETE /friends/{userId}` · `POST /friends/{userId}/accept` · `POST /friends/{userId}/reject`	токен
	`GET /me/friends` · `GET /me/friend-requests` · `GET /me/friend-search?email=`	токен
Подписки	`POST /organizations/{id}/subscription` · `DELETE /organizations/{id}/subscription` · `GET /me/subscriptions`	токен
Рекомендации	`GET /recommendations` (`lat`, `lng`, `limit`)	токен
	`POST /recommendations/feedback` (`CLICK` или `DISMISS`) · `DELETE /recommendations/feedback/{eventId}`	токен
Admin	`GET /admin/organizations` · `POST /admin/organizations/{id}/approve` · `…/reject`	`ROLE_ADMIN`
	`GET /admin/events` (с результатом автопроверки, помеченные ⚠ — первыми) · `POST /admin/events/{id}/unpublish` · `…/publish`	`ROLE_ADMIN`
	`POST /admin/events/{id}/moderation/reviewed` — снять пометку ⚠	`ROLE_ADMIN`
	`GET /admin/comments`	`ROLE_ADMIN`

Мониторинг

`GET /actuator/health` (с состоянием БД), `/actuator/info`, `/actuator/metrics` — публичны.

ИНЖЕНЕРИЯ

Конструктор мероприятий

Создание мероприятия (`/events/new`) собирается из блоков, а формат мероприятия описывается **данными, а не кодом**.

Блоки формы

1. **Шаблон** (`GET /api/v1/templates`): «Тренировка», «Игра / матч», «Турнир», «Забег» (только для организаций), «Интеллектуальная игра», «Поход», «Мастер-класс» (не спорт) и «Свободный формат». Шаблон задаёт значения по умолчанию (вместимость, режим записи, уровень, длительность), набор дополнительных полей и вопросы участнику.
2. **Вид спорта.** Выбор из списка или «+ Свой вид спорта». Название нормализуется в slug (`Воркаут` → `vorkaut`) и сразу попадает в общий каталог и фильтры; дублей не создаётся.
3. **Свои параметры.** Произвольные пары «название — значение» поверх шаблона.
4. **Анкета участника.** Организатор сам добавляет вопросы (текст, число, выбор, да или нет), на которые отвечает подающий заявку.
5. **Данные организатора** (только для организаций; контакты обязательны): контактное лицо, телефон и e-mail, сайт или регламент, площадка, медицинская справка и страховка, список документов, призы, программа, инфраструктура, условия отмены, партнёры. Если заданы требования, участник подтверждает ознакомление при подаче заявки.
6. **Место и время.** Точка выбирается кликом по карте; время вводится по местному времени площадки (город и часовой пояс определяются по точке).

Публикует мероприятие пользователь — от своего имени; организатор — от имени организации, только после одобрения администратором (иначе `403`).

Типы полей

`TEXT`, `TEXTAREA`, `NUMBER`, `SELECT`, `MULTISELECT`, `BOOLEAN`. Каждое поле описывается `FieldDef`:

Атрибут	Смысл
`key`	Идентификатор: латиница, цифры и `_`, начинается с буквы, до 40 символов; в рамках формы уникален
`label`	Название до 100 символов
`type`	Один из шести типов
`required`	Обязательность
`options`	Варианты для `SELECT` и `MULTISELECT`: от 1 до 20, без пустых и повторов
`unit`, `placeholder`, `help`	Единица измерения, подсказка, пояснение

Валидатор

`FieldValidator` проверяет и описания полей (то, что определил автор), и значения, введённые по этим описаниям. Поэтому формы мероприятия и заявки полностью управляются данными.

- Не более 12 полей в форме; вариант ответа не длиннее 100 символов.

- Ограничения длины: текст до 500 символов, длинный текст до 3000.
- Значения очищаются и приводятся к типу поля: пустые отбрасываются, числа (в том числе строки с запятой вместо точки) становятся числом по модулю не больше миллиарда, для списков допустимы только объявленные варианты.
- Неизвестный ключ в значениях — ошибка; обязательное поле без значения — ошибка; обязательное поле `BOOLEAN` должно быть отмечено.
- Ошибки возвращаются как `400` с понятным сообщением, например «Поле «Размер»: недопустимый вариант».

Снимок схемы

Поля шаблона копируются в мероприятие снимком (`events.fields`). Правка шаблона не ломает уже созданные события. Значения полей хранятся в `events.details`, свои параметры — в `events.custom_fields`, анкета — в `events.registration_fields`, сведения организации — в `events.org_details`.

Как добавить новый формат

Новый формат — одна строка в таблице `event_templates`: `code`, название, значки, `audience` (`ANY` или `ORGANIZATION`), JSON-схемы `fields` и `registration_fields`, значения по умолчанию. Правок кода и релиза не требуется.

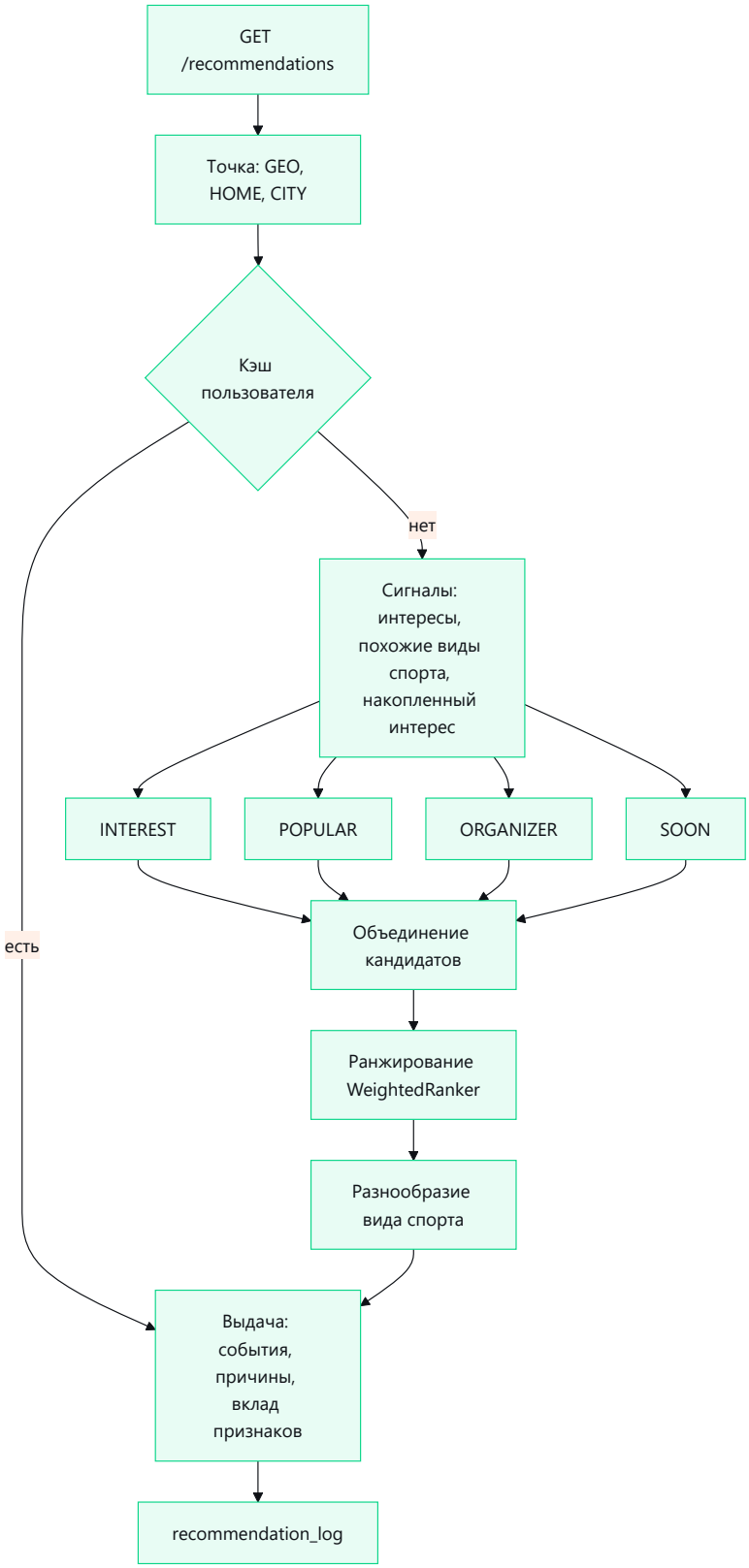
ИНЖЕНЕРИЯ



Рекомендательная система

Страница **«Для вас»** (`/recommended`) подбирает мероприятия по интересам, месту, времени и истории пользователя. Каждая карточка показывает причины, а кнопка «Не интересно» скрывает событие и понижает похожие (с возможностью отмены).

Рекомендации строятся внутри бэкенда на данных PostgreSQL + PostGIS, без отдельного поискового кластера: данные всегда свежие (свободные места, статус, заявки), нет синхронизации индекса.



1. Кандидаты

Каждый из четырёх источников делает один запрос к GiST-индексу и берёт не больше `candidates-per-source` событий (по умолчанию 300) — поэтому даже в плотном городе на ранжирование попадает ограниченное число строк.

Источник	Что отбирает
`INTEREST`	Виды спорта из интересов, похожие виды и те, куда пользователь уже ходит; ближайшие
`POPULAR`	Самое популярное рядом (предрассчитанная `mv_event_popularity`)
`ORGANIZER`	События организаторов и клубов, к которым пользователь уже ходил
`SOON`	Ближайшее по времени рядом — разведка и запасной вариант

Жёсткие фильтры для всех источников: событие опубликовано; начинается в пределах горизонта (30 дней); регистрация открыта; в радиусе пользователя; не своё мероприятие и нет активной или отклонённой заявки; не скрыто кнопкой «Не интересно»; подходит по бюджету и возрасту (если указаны). Мероприятия без свободных мест отсеиваются при ранжировании.

Точка поиска: геолокация устройства → домашняя точка → центр домашнего города.

2. Ранжирование

`WeightedRanker`, модель `weighted-v1`: взвешенная сумма нормированных признаков минус штраф за отвергнутое.

Признак	Вес	Смысл
`interest`	0,35	Явный интерес; для похожих видов — частичный балл
`distance`	0,20	Экспоненциальное затухание с расстоянием
`affinity`	0,15	Куда пользователь реально ходит (вид спорта, формат)
`urgency`	0,10	Чем скорее начало, тем выше
`skill`	0,08	Совпадение уровня подготовки
`social`	0,07	Популярность и заполненность
`organizer`	0,05	Знакомый организатор
`time`	0,05	Удобное время — по местному времени площадки
`dislike-penalty`	−0,30	Штраф за виды спорта и организаторов, от которых отказывались

Для каждого элемента формируется до трёх причин («Вы интересуетесь: Бег», «1,4 км от вас», «Завтра в 07:00», «Тем, кто любит Футбол, нравится Волейбол») и словарь `breakdown` с вкладом каждого признака.

Разнообразие. В выдаче не больше двух событий одного вида спорта подряд (`max-same-sport-in-row`).

3. Накопленные сигналы

События живут несколько дней, поэтому система учится на устойчивых сущностях — виде спорта, шаблоне, организаторе, времени суток (`user_affinities`). Действия обновляют вес мгновенно: запись `+1`, отмена `-0,5`, «Не интересно» `-0,5`, клик `+0,15`. Значения затухают экспоненциально, период полураспада — 90 дней.

Похожие виды спорта. Материализованное представление `mv_sport_similarity` считает косинусное сходство видов по совместной встречаемости у одних и тех же пользователей.

4. Холодный старт

Без интересов и истории выдача строится на `POPULAR` и `SOON`, а клиент приглашает указать интересы. Если неизвестно и местоположение, ответ содержит `needsLocation = true`.

5. Кэш и фоновые задачи

- Готовая выдача кэшируется на пользователя и точку (с точностью около 1 км) на 2 минуты и сбрасывается при записи, отмене, решении организатора, «Не интересно» и смене предпочтений. Реализация `InMemoryRecommendationCache` скрыта за интерфейсом `RecommendationCache`.
- `RecommendationJobs`: популярность обновляется каждые 5 минут; ночью (03:30) пересчитываются похожие виды спорта, пересобираются сигналы из истории и создаются секции журнала на текущий и следующий месяц. Каждая задача берёт `pg_try_advisory_xact_lock`, поэтому при нескольких экземплярах бэкенда выполняется ровно в одном.

6. Журнал

`recommendation_log` хранит показы (`IMPRESSION`, позиция, балл, модель, `request_id`), клики, отказы и записи. Это источник метрик (CTR, доля записавшихся) и обучающая выборка для следующей версии ранжирования.

Интересы

Интересы задаются через `PUT /api/v1/me/preferences`: вид спорта, город, радиус, уровень, удобное время, бюджет, год рождения. В интерфейсе их собирает блок «Мои интересы» в профиле и кнопка «Мои интересы» на странице рекомендаций — это одна и та же форма. Регистрация интересы не запрашивает.

Настройка без пересборки

`application.yml`, секция `recommendations`:

```
recommendations:
  default-radius-km: 25
  horizon-days: 30
  candidates-per-source: 300
  cache-ttl: PT2M
  max-same-sport-in-row: 2
  affinity-half-life-days: 90
  weights: { interest: 0.35, distance: 0.20, affinity: 0.15, urgency: 0.10,
             skill: 0.08, social: 0.07, organizer: 0.05, time: 0.05, dislike-penalty: 0.30 }
  jobs: { enabled: true, popularity-refresh: PT5M, nightly-cron: "0 30 3 * * *" }
```

Точки расширения

Интерфейс	Что даёт
`CandidateSource`	Новый источник кандидатов — ещё один бин
`Ranker`	Другая модель ранжирования; имя модели пишется в журнал для сравнения вариантов
`RecommendationCache`	Кэш в Redis при работе нескольких экземпляров бэкенда

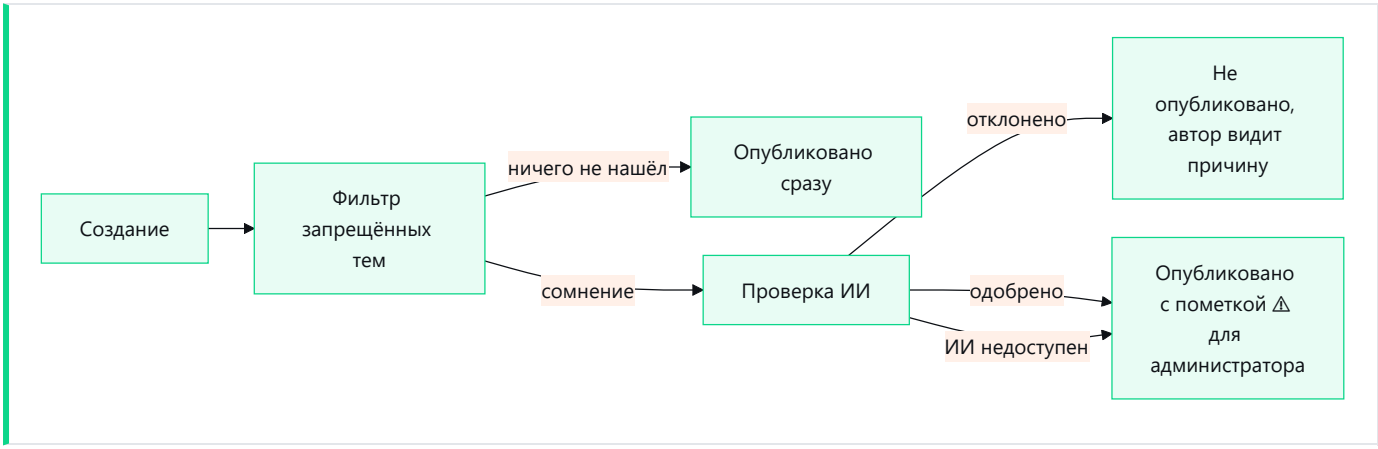
Почему не Elasticsearch и не обученная модель

- **Без Elasticsearch.** Для рекомендаций важны свежие данные и геопоиск, а не полнотекстовый поиск; индекс отставал бы от БД. Если понадобится текстовый поиск на всю страну, его можно добавить отдельно.
- **Формула вместо LTR.** Реальных данных для обучения пока нет, а модель, обученная на сгенерированных данных, лишь воспроизвела бы формулу. Журнал и интерфейс `Ranker` готовы: обученная модель подключается заменой бина, когда накопится трафик.

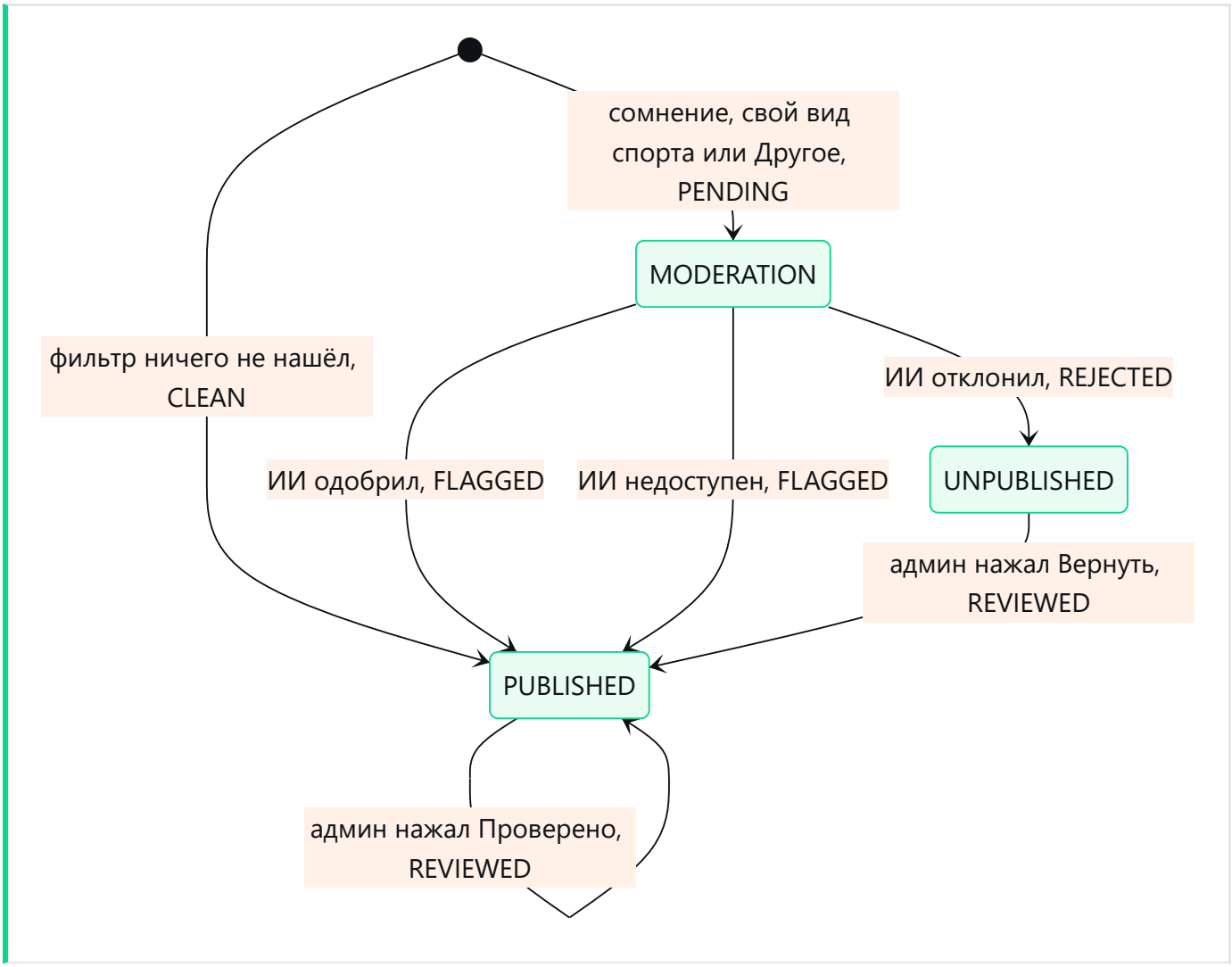
ИНЖЕНЕРИЯ

Автопроверка мероприятий

Каждое новое мероприятие проверяется автоматически. Для автора это несколько секунд после нажатия «Опубликовать»: кнопка показывает «Публикуем и проверяем…».



Статусы



Пока идёт проверка, мероприятие скрыто (статус `MODERATION`) и нигде не показывается.

1. Фильтр запрещённых тем

`BannedWordFilter` ищет по словарю десять категорий: политика, алкоголь, табак и вейпы, наркотики и допинг, азартные игры и ставки, дискриминация и оскорбления, насилие и экстремизм, контент 18+, финансовые схемы и спам, нецензурная лексика.

Проверяются все текстовые поля (`ModerationText`): название, описание, вид спорта, параметры шаблона, свои поля, вопросы участникам, сведения организации, адрес. Текст нормализуется в трёх вариантах: как есть; с заменой латинских и цифровых двойников на кириллицу («водка» латиницей, «в0дка»); со склейкой букв, разделённых точками, дефисами и пробелами («в.о.д.к.а», «в о д к а»). Не больше 20 находок на мероприятие.

Фильтр не выносит приговор, а только поднимает сомнение. Всегда уходят на проверку ИИ мероприятия со **своим видом спорта** и в категории **«Другое»**: словарь не может оценить их тематику.

2. Проверка ИИ

`RouterAiModerationClient` обращается к RouterAI — OpenAI-совместимому шлюзу.

Параметр	Значение
Запрос	`POST https://routerai.ru/api/v1/chat/completions`
Модель по умолчанию	`google/gemini-2.5-flash-lite` — быстрая, недорогая, хорошо понимает русский
Параметры	`temperature 0`, `response_format: json_object`, `max-tokens 200`
Таймаут	8 секунд
Вход	До 4000 символов текста мероприятия плюс находки фильтра как подсказка

Системный промпт описывает тематику платформы и критерии отказа. Модель решает, относится ли мероприятие к спорту, активности или ЗОЖ и нет ли нарушений; упоминание в запрещающем смысле («без алкоголя») нарушением не считается. Текст мероприятия передаётся между тегами как данные, а не инструкции (защита от prompt injection). Ответ — строго JSON `{"decision", "reason"}`; разбор терпим к обёрткам вроде Markdown-блоков, иначе считается, что ИИ недоступен.

3. Итог

- **Отклонено ИИ.** Статус `UNPUBLISHED` с причиной «Не прошло автоматическую проверку: …». Автор видит её сразу на форме и в «Моих событиях». Ошибочное отклонение администратор исправляет кнопкой «Вернуть».
- **Одобрено ИИ.** Мероприятие публикуется, но в `/admin` помечается знаком ⚠ с объяснением модели, найденными словами и именем модели. Администратор нажимает «Проверено» или снимает мероприятие.
- **ИИ недоступен** (нет ключа, таймаут, сбой). Добросовестный автор не ждёт: мероприятие публикуется с пометкой ⚠ «нужна ручная проверка».

Надёжность

- Проверка ИИ выполняется **вне транзакции**: база не блокируется на время ожидания. Решение применяется отдельной короткой транзакцией с условием «всё ещё `PENDING`», поэтому повторный вызов безопасен.
- Ответ `POST /events` уже содержит итог проверки.
- Если бэкенд перезапустился посреди проверки, `ModerationJobs` раз в минуту доделывает «зависшие» (старше `stuck-after`, по умолчанию 1 минута).

Настройки

`application.yml`, секция `moderation`:

Параметр	Переменная	По умолчанию
`enabled`	`MODERATION_ENABLED`	`true`
`review-custom-sports`	—	`true`
`review-sport-slugs`	—	`[drugoe]`
`stuck-after`	—	`PT1M`
`llm.base-url`	`ROUTERAI_BASE_URL`	`https://routerai.ru/api/v1`
`llm.api-key`	`ROUTERAI_API_KEY`	пусто — второй этап пропускается

Параметр	Переменная	По умолчанию
`llm.model`	`MODERATION_LLM_MODEL`	`google/gemini-2.5-flash-lite`
`llm.timeout`	—	`PT8S`

Модель можно заменить любой из каталога RouterAI без изменения кода.

ИНЖЕНЕРИЯ

Локальный запуск

Всё в Docker

Нужен запущенный **Docker Desktop**. В корне проекта:

```
# 1. (необязательно) ключи — без них приложение работает с заглушками
cp .env.example .env
#    впишите VITE_2GIS_KEY (https://platform.2gis.ru) и ROUTERAI_API_KEY

# 2. Сборка и запуск базы, Redis, бэкенда и веб-клиента
docker compose up --build -d

# 3. Проверка состояния
docker compose ps
```

Ключ 2GIS встраивается во фронтенд **при сборке**, поэтому после его изменения нужна пересборка: `docker compose up --build -d`.

Доступные сервисы

Сервис	Ссылка	Описание
Веб-приложение	`http://localhost:3000`	Карта, каталог, конструктор, рекомендации, профили, модерация
Swagger UI	`http://localhost:8080/swagger-ui/index.html`	Интерактивная документация OpenAPI (и через порт 3000)
OpenAPI JSON	`http://localhost:8080/v3/api-docs`	Спецификация для генерации клиентов
Healthcheck	`http://localhost:8080/actuator/health`	Состояние бэкенда и базы данных
PostgreSQL	`localhost:5432`	БД `sportikus`, пользователь `sportikus_user`

Сброс данных

Схема и демо-данные создаются миграциями при старте. После обновления схемы или демо-данных пересоздайте базу и загрузки:

```
docker compose down -v && docker compose up --build -d
```

Данные хранятся в двух volume: `postgres_data` (база) и `profile_uploads` (фото профилей и логотипы). Обычный `docker compose up -d --build` данные не стирает.

Разработка без Docker

Для бэкенда нужны **Java 21**, **Maven 3.9+** и PostgreSQL **с расширением PostGIS**: обычный PostgreSQL не подойдёт (миграция выполняет `CREATE EXTENSION postgis`). Проще всего поднять из compose только базу и необязательный Redis:

```
docker compose up -d postgres redis
```

```
# Backend, http://localhost:8080
cd backend
mvn spring-boot:run
```

```
# Frontend, http://localhost:5173
cd frontend
npm install
npm run dev
```

Vite проксирует запросы `/api` на `http://localhost:8080`. Ключ 2GIS берётся из `.env` в корне проекта или из `frontend/.env` (приоритетнее).

Переменные окружения

Полный пример — `.env.example`.

Переменная	Где используется	По умолчанию
`VITE_2GIS_KEY`	Сборка фронтенда	пусто (вместо карты заглушка)
`VITE_2GIS_STYLE_ID`, `VITE_2GIS_STYLE_ID_DARK`	Сборка фронтенда	пусто (стандартный стиль 2GIS)
`VITE_API_URL`	Фронтенд	`/api/v1`
`SPRING_DATASOURCE_URL`, `_USERNAME`, `_PASSWORD`	Backend	`jdbc:postgresql://localhost:5432/sportikus`, `sportikus_user`, `sportikus_password`
`REDIS_HOST`, `REDIS_PORT`	Backend	`localhost`, `6379`
`MAP_CACHE_ENABLED`	Backend	`true`
`JWT_SECRET`	Backend	значение для разработки — **замените в реальном окружении**
`UPLOAD_DIR`	Backend	`uploads` (в Docker `/app/uploads`)

Переменная	Где используется	По умолчанию
`ROUTERAI_API_KEY`	Backend (автопроверка)	пусто: работает только фильтр, сомнительное помечается ⚠
`MODERATION_LLM_MODEL`	Backend (автопроверка)	`google/gemini-2.5-flash-lite`
`MODERATION_ENABLED`	Backend (автопроверка)	`true`

Файл `.env` не попадает в git. Ключи в репозиторий не коммитятся.

Эта документация

Сайт документации и PDF собираются из каталога `docs-zensical/`. Инструкции — в `docs-zensical/README.md`.

ИНЖЕНЕРИЯ

Тестирование

Команды

```
cd backend && mvn test            # юнит-тесты бэкенда
cd frontend && npm run test       # автотесты фронтенда (Vitest)
cd frontend && npm run build      # проверка типов (tsc) и сборка
cd frontend && npm run lint       # oxlint
```

Backend

JUnit 5 и AssertJ. Тестовые классы:

Класс	Что проверяет
`FieldValidatorTest`	Валидатор полей конструктора: описания и значения
`SlugUtilTest`	Нормализация названия вида спорта в slug
`ParticipationServiceTest`	Машина состояний участия: запись, заявка, лимит мест, решения организатора, отмена
`WeightedRankerTest`	Ранжирование рекомендаций: интересы, расстояние, штрафы, местное время, разнообразие
`BannedWordFilterTest`	Срабатывания фильтра, обходы через латиницу и разделители, отсутствие ложных срабатываний на обычных спортивных текстах
`RouterAiModerationClientTest`	Разбор ответа ИИ
`EventModerationServiceTest`	Решения «одобрить», «отклонить», «ИИ недоступен»
`FriendshipServiceTest`	Заявки в друзья, принятие, встречные заявки

Класс	Что проверяет
`MapCacheKeyTest`	Нормализация фильтра и ключ кэша карты
`RedisMapCacheTest`	Кэш карты: версии, промахи, устойчивость к ошибкам хранилища

Frontend

Vitest, Testing Library и jsdom (`vitest.config.ts`, `src/test/setup.ts`). На момент написания — 14 файлов и 139 тестов.

Область	Файлы
Чистая логика	`utils/format`, `utils/eventView`, `utils/moderation`, `utils/eventsSnapshot`
Конструктор форм	`FieldInput`, `TagInput`, `CustomFieldsEditor`, `QuestionsEditor`
Карточки и статусы	`EventCard`, `MapEventCard`, `EventBits`, `ParticipationBadge`
Диалог записи	`JoinDialog`: обязательные вопросы, требования организатора, ошибки сервера
Регрессия	`CreateEventPage`: выбор режима «свободный вход» не ломает страницу

Тесты, которые касаются 2GIS (`LocationPicker`) или сети (`api`, `AuthContext`), подменяют эти модули: SDK карты в jsdom не загружается.

> **О регрессионном тесте**
>
> Баг «белый экран при выборе свободного входа» вызывал сдвиг прокрутки всего документа из-за визуально скрытого радио-поля без позиционированной обёртки. jsdom не умеет считать раскладку и прокрутку, поэтому тест проверяет, что страница не падает, радио-группа переключается и у обёртки есть класс `relative` — то есть сам исправленный контракт разметки, а не пиксели.

Чего нет

Нет сквозных браузерных тестов (Playwright). Сценарии Т3 проверяются вручную по User flows; интеграционных тестов с реальной PostgreSQL в репозитории пока нет, поэтому SQL геопоиска и рекомендаций проверялся на запущенном стеке.

ИНЖЕНЕРИЯ

Ограничения и развитие

Не реализовано

- PWA (манифест, service worker, офлайн) и push-уведомления.
- Уведомления об ответах в обсуждении.
- Сбор интересов при регистрации: их указывают в профиле (`/profile`) или на странице рекомендаций.
- «Поделиться геопозицией»: в конфигурации есть флаг `features.location-sharing.enabled`, кода под ним пока нет.

- Сквозные браузерные тесты и интеграционные тесты с реальной базой.

Известные упрощения

Что	Как сейчас	Что менять при росте нагрузки
CORS	Открыт для всех источников (для демо)	Ограничить список источников
Загрузки	Лежат на диске (volume)	Объектное хранилище при нескольких экземплярах бэкенда
Кэш рекомендаций	Локален процессу	Redis через интерфейс `RecommendationCache`
Обсуждение	Опрос раз в 8 секунд	SSE или WebSocket
Счётчики мест на карте	Могут отставать до 60 секунд (TTL кэша карты)	Сбрасывать кэш при изменении участия или обновлять счётчики поверх кэша
Достижения	Считаются SQL-запросами при каждом обращении; клиент опрашивает раз в 15 секунд	Предрасчёт и событийная модель
Похожие виды спорта	Порог поддержки мал из-за демо-данных	На реальном трафике поднять `similarity-min-cooccurrence`
Ключ 2GIS	Встраивается при сборке фронтенда	Серверный прокси или ограничение ключа по домену

Направления развития

- Обученная модель ранжирования (LTR) на журнале показов `recommendation_log`.
- Отдельный полнотекстовый поиск на всю страну.
- Выделение рекомендаций в сервис при росте нагрузки.
- Уведомления и реальное время для обсуждений и заявок.
- Мобильное приложение поверх существующего REST API.

Глоссарий

Термин	Значение
Мероприятие (событие)	Спортивная или иная активность с местом, временем, вместимостью и режимом записи
Шаблон	Готовый формат мероприятия: поля, вопросы участнику и значения по умолчанию, описанные данными в таблице `event_templates`
Конструктор	Форма создания мероприятия, собранная из шаблона, видов спорта, своих параметров и анкеты
Анкета участника	Вопросы организатора к подающему заявку (`registration_fields`)
Режим записи	`OPEN` — запись подтверждается сразу; `APPROVAL` — по заявке с решением организатора
Заявка	Участие в статусе `PENDING`, ожидающее решения организатора
Участие	Связь пользователя с мероприятием; статусы `PENDING`, `CONFIRMED`, `REJECTED`, `CANCELLED`
Билет	Уникальный код участия `SPT-XXXXXXXX`, отображаемый QR-кодом
Отметка на входе (check-in)	Подтверждение организатором, что билет предъявлен и использован
Организатор	Автор мероприятия или владелец организации, от имени которой оно опубликовано
Организация	Юридическое лицо с реквизитами (ИНН, ОГРН, КПП); публикует после одобрения администратором
Автопроверка	Фильтр запрещённых тем и при сомнении проверка ИИ при создании мероприятия
Пометка ⚠	Знак для администратора: мероприятие опубликовано, но требует внимания
Мягкое скрытие	Сообщение остаётся в базе, обычным пользователям показывается заглушка
Архив	Статус `ARCHIVED`: завершённое организатором мероприятие
Рекомендация	Мероприятие в выдаче «Для вас» с причинами и вкладом признаков
Источник кандидатов	Способ выбрать мероприятия для ранжирования: `INTEREST`, `POPULAR`, `ORGANIZER`, `SOON`
Накопленный интерес (affinity)	Вес сущности (вида спорта, организатора, шаблона, времени суток), растущий от действий пользователя и затухающий со временем
Холодный старт	Ситуация без интересов и истории: показывается популярное рядом
PostGIS	Расширение PostgreSQL для геоданных и пространственных индексов
GiST	Тип индекса для геопоиска по близости
Кэш карты	Список мероприятий области карты без личных данных, хранится в Redis
RouterAI	OpenAI-совместимый шлюз к языковым моделям, через который работает проверка ИИ
2GIS MapGL	JavaScript-библиотека карт 2GIS, используемая клиентом
JWT	Токен авторизации; передаётся в заголовке `Authorization: Bearer`

Термин	Значение
Flyway	Инструмент версионирования схемы базы данных
Zensical	Генератор статических сайтов документации, на котором собрана эта документация